COLAB

1. Analisi concettuale

1.1. Diagramma UML delle classi

Specifica dei tipi di dato

Operazioni ausiliarie

non_sovrapposti(

sovrapposti_Ora(

secondi_per_unita_misura(um: UnitaMisura): Intero > 0

Specifica dei Vincoli Esterni

[Utente.AccessoSpazio.non_sovraposti]

[Utente.AccessoSpazio.max_1_accesso_senza_uscita]

[Utente.UsoServizio_incluso_TipoAbbonamento]

[Utente.Prenota_nel_periodo_validita_suo_abbonamento]

[Postazione.Utente_uso_esclusivo]

[Storico.inizio_fine_inclusi_in_durata_abbonamento]

[AccessoSpazio.Utente_accede_solo_con_abbonamento_valido]

[AcessoSpazio.Entrata_minore_Uscita]

[UsoServizio.Inizio_minore_Fine]

[UsoServizio.Quantita_valorizzata_se_fine_valorizzato]

[UsoServizio.inizio_fine_inclusi_in_AccessoSpazio_esistente]

[Prenotazione.Inizio_minore_Fine]

[Prenotazione.istante_canc_valorizzata_se_cancellata=True]

[Servizio.se_prenotabile=True_allora_Prenotazione_obbligatoria_prima_UsoServizio]

[Servizio.uso_esclusivo_durante_prenotazione]

[TipoAbbonamento.Inizio_iscrizione_minore_Fine_iscrizione]

[TipoAbbonamento.num_Utenti_associati_Abbonamento_minoreUguale_max_num_utenti]

Operazioni di classe UsoServizio

Operazioni ausiliarie

calcolo_quantita_da_tempo(): Intero > 0

Operazioni di classe

prezzo(): Reale >= 0

Operazioni di classe Abbonamento

Operazioni ausiliarie

ab_quantita_serv_utilizata_in_mese_entro_data(serv: Servizio, t: DataOra): Intero >= 0

ab_quantita_serv_utilizata_in_mese(serv: Servizio, t: DataOra): Intero >= 0

ab_quantita_serv_gratis_rimasta(serv: Servizio, t: DataOra): Intero >= 0

Operazioni di classe

fine(): DataOra

utenti(): Utente [0..*]

1.2. Diagramma UML degli Use-Case

Operazioni di Use-Case Statistica di utilizzo servizi

media_giornaliera_utenti_nei_periodi(

costo_totale_servizi_oltre_soglia_gratis_nel_periodo(

servizi_popolari_nel_periodo(

Operazioni di Use-Case Strumenti monitoraggio servizi utilizzati

utenti_associati_abbonamento_senza_entrate(

servizi_utilizzati(

Operazioni di Use-Case Strumenti utilizzo servizi

chiusura_UsoServizio(

registrazione_accesso_spazio(ut: Utente): AccessoSpazio

registrazione_uscita_spazio(asp: AccessoSpazio, ut: Utente)

Operazioni di Use-Case Strumenti ricerca tipo abbonamento

ricerca_TipoAbbonamento_attuale_con_Servizio(serv: Servizio) : TipoAbbonamento [0..*]

Operazioni di Use-Case Strumenti sottoscrizione abbonamenti

sottoscrizione_abbonamento_con_TipoAbbonamento(cl: Cliente, tipab: TipoAbbonamento, U: Utenti[1..*])

Operazioni di Use-Case Strumenti monitoraggio accessi

verifica_abbonamento_valido(

Operazioni di Use-Case Strumenti di creazione e gestione servizi, tariffe

creazione_TipoAbbonamento(

creazione_tariffa(

2. Ristrutturazione per Basi Dati

2.1. Diagramma UML ristrutturato per bd

2.2. Specifica dei vincoli esterni

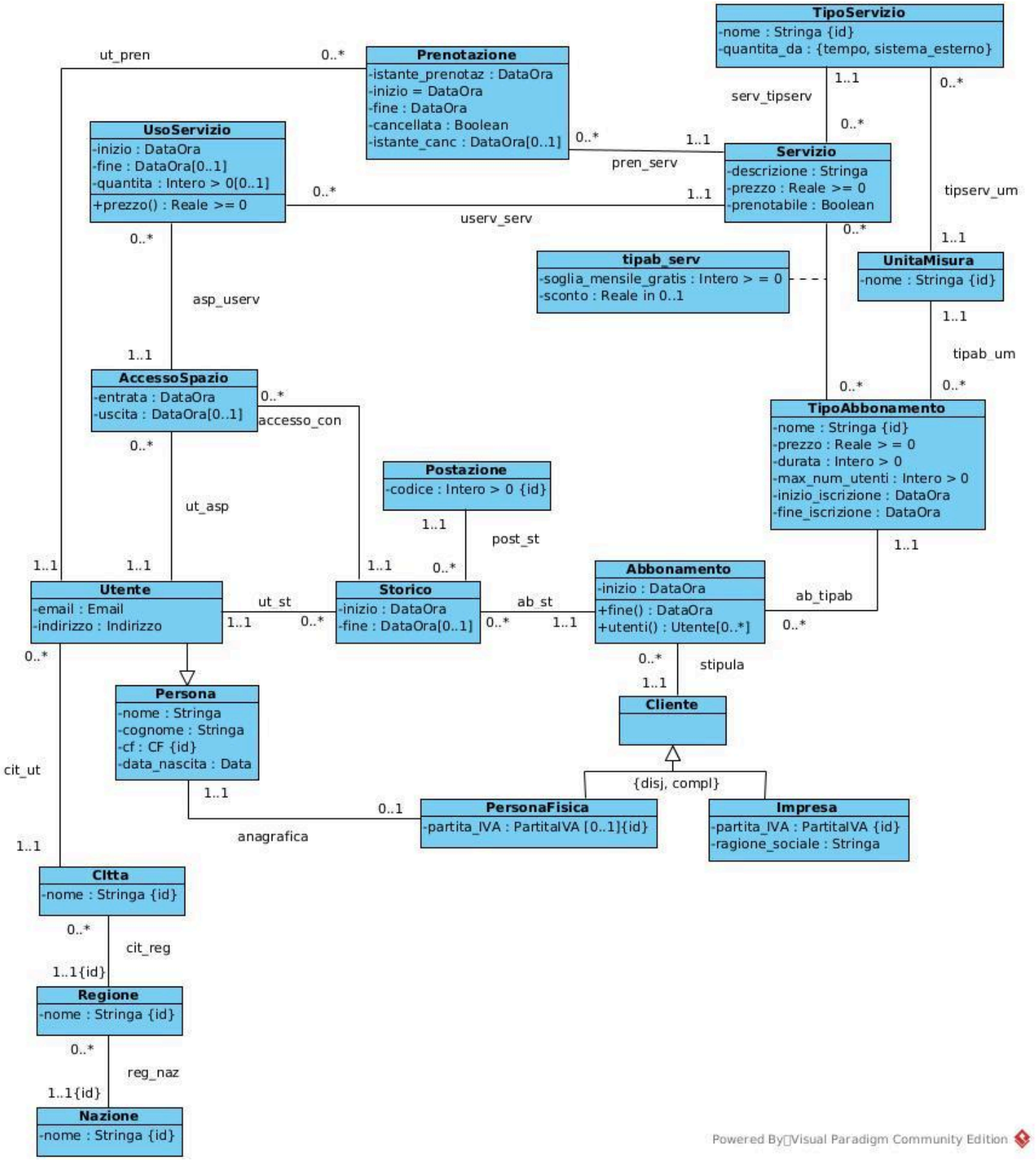
[Cliente.Disgiunzione]

[Cliente.Completezza]

2.3. Traduzione diretta

1. Analisi concettuale

1.1. Diagramma UML delle classi



Ho provato a modellare il sistema in modo che permette agli utenti di avere più abbonamenti assegnati.
Esempio:
Una Persona (Utente) usufruisce di abbonamento_1 aziendale durante i giorni lavorativi.
Stessa Persona come PersonaFisica puo stipulare un altro abbonamento_2 (per uso personale o no).
Nel caso se lo stipula per uso personale PersonaFisica diventa anche Utente e puo usare il sistema con abbonamento_2 durante fine settimana.
Sistema riconosce Utente e offre i servizi in base al abbonamento con cui è stato effettuato accesso.

Per non duplicare i dati anagrafici della PersonaFisica ho aggiunto associazione verso classe Persona.

Ho aggiunto la classe Prenotazione anche se i requisiti non lo chiedevano. Capisco che nella vita reale si dovrebbe modellare solo cio che viene richiesto, ma qui ho provato a gestire anche questa opzione.

Specifica dei tipi di dato

Tipo Partitalva: Stringa di 11 caratteri numerici secondo lo standard

Tipo Email: Stringa secondo lo standard

Tipo Indirizzo: tipo di record con campi

via: Stringa

civico: Stringa

cap: Stinga numerica di lunghezza 5 caratteri

Tipo CF: Stringa secondo lo standard

Tipo Orario: tipo record con campi

ora_inizio: Ora

ora_fine: Ora

Operazioni ausiliarie

non_sovrapposti(

in1: DataOra,
fin1: DataOra,
in2: DataOra,
fin2: DataOra
) : Boolean

precondizione:

- *in1 < fin1 e in2 < fin2*

postcondizione:

- operazione non fa side-effect
- *result* = True se e solo se fin1 < in2 oppure fin2 < in1

Pre: $< (in_1, fin_1) \wedge \; < (in_2, fin_2)$

Post: $sovrapposti_DataOra(in_1, fin_1, in_2, fin_2, True) \Leftrightarrow \; < (fin_1, in_2) \vee \; < (fin_2, in_1)$

→[Utente.AccessoSpazio.non_sovraposti]
→[Postazione.Utente_uso_esclusivo]

sovrapposti_Ora(

in1: Ora,
fin1: Ora,
in2: Ora,
fin2: Ora
): Boolean

precondizione:

- *in1 < fin1* e *in2 < fin2*

postcondizione:

- operazione non fa side-effect
- *result* = True se e solo se *in1 < fin2* **e** *in2 < fin1*

Pre: $< (in_1, fin_1) \wedge \; < (in_2, fin_2)$

Post: $sovrapposti_Ora(in_1, fin_1, in_2, fin_2, True) \Leftrightarrow \; < (in_1, fin_2) \wedge \; < (in_2, fin_1)$

→ media_giornaliera_utenti_nei_periodi(.)

secondi_per_unita_misura(um: UnitaMisura): Intero > 0

Precondizioni:

- *um.nome* ∈ {secondo, minuto, ora, giorno, settimana}

Postcondizione:

- operazione non fa side-effect
- viene restituiro valore "result" definito come segue
  - se *um.nome* = secondo, *result* = 1
  - se *um.nome* = minuto, *result* = 60
  - se *um.nome* = ora, *result* = 3600
  - se *um.nome* = giorno, *result* = 86400
  - se *um.nome* = settimana *result* = 604800

→ calcolo_quantita_da_tempo(): Intero

Specifica dei Vincoli Esterni

[Utente.AccessoSpazio.non_sovraposti]

Per ogni asp1: AccessoSpazio e asp2: AccessoSpazio e ut: Utente tali che

(ut, asp1): ut_asp e (ut, asp2): ut_asp e asp1 ≠ asp2

deve essere vero che

- non_sovraposti(asp1.entrata, asp1.uscita, asp2.entrata, asp2.uscita) = *True*

$$\forall ut, asp_1, asp_2 \; (Utente(ut) \wedge AccessoSpazio(as_1) \wedge AccessoSpazio(as_2)$$
$$\wedge \; ut_asp(ut, asp_1) \wedge ut_asp(ut, asp_2) \wedge asp_1 \neq asp_2$$
$$\wedge \; ((\exists \, entr_1, usc_1, entr_2, usc_2 \, (entrata(as_1, entr_1) \wedge uscita(as_1, usc_1)$$
$$\wedge \; entrata(as_2, entr_2) \wedge uscita(as_2, usc_2)$$
$$\Rightarrow non_sovraposti(entr_1, usc_1, entr_2, usc_2 , True))$$

[Utente.AccessoSpazio.max_1_accesso_senza_uscita]

Per ogni ut: Utente

non deve esiste asp1: AccessoSpazio e asp2: AccessoSpazio tali che

- *(ut, asp1): ut_asp* e *(ut, asp2): ut_asp* e *asp1 ≠ asp2* e
- *asp1.uscita* non è valorizzato e *asp2.uscita* non è valorizzato

$$\forall ut \; (Utente(ut) \Rightarrow$$
$$\neg \exists \, asp_1, asp_2 \; (AccessoSpazio(asp_1) \wedge AccessoSpazio(asp_2) \wedge ut_asp(ut, asp_1) \wedge ut_asp(ut, asp_2) \wedge asp_1 \neq asp_2$$

$\wedge\ (\exists\, entr_1, \exists\, entr_2\ (entrata(as_1,\ entr_1)\ \wedge\ (entrata(as_2,\ entr_2))$

$\wedge\ \neg\exists\, usc_1\ (\ uscita(as_1,\ usc_1)\ \wedge\ \neg\exists\, usc_2\ (\ uscita(as_2,\ usc_2)\ ))$

[Utente.UsoServizio_incluso_TipoAbbonamento]

Per ogni ut: Utente, asp: AccessoSpazio, userv: UsoServizio, serv: Servizio, st: Storico, ab: Abbonamento, tipab: TipoAbbonamento,

tali che

- (ut, asp): ut_asp e (asp, userv): asp_userv e (userv, serv): userv_serv e
- (ut, st): ut_st e (ab, st): ab_st e (ab, tipab): ab_tipab

deve essere vero che

- esiste (tipab, serv): tipab_serv

$\forall\, ut,\ asp,\ userv, st,\ ab,\ tipab,\ serv$

$(Utente(ut)\ \wedge\ AccessoSpazio(asp)\ \wedge\ UsoServizio(userv)$

$\wedge\ Storico(st)\ \wedge\ Abbonamento(ab)\ \wedge\ TipoAbbonamento(tipab)\ \wedge\ Servizio(serv)$

$\wedge\ \ ut_asp(ut,\ asp)\ \wedge\ asp_userv(asp,\ userv)\ \wedge\ userv_serv(userv,\ serv)$

$\wedge\ ut_st(ut,\ st)\ \wedge\ st_ab(st,\ ab)\ \wedge\ ab_tipab(ab,\ tipab)\ )$

$\Rightarrow\ \exists\, tipab_serv(tipab,\ serv)$

[Utente.Prenota_nel_periodo_validita_suo_abbonamento]

Per ogni ut: Utente, pren: Prenotazione, serv: Servizio

tali che

- (ut, pren): ut_pren e (pren, serv): pren_serv

deve essere vero che

- esiste st: Storico, ab:Abbonamento, tipab: TipoAbbonamento tali che
  - (ut, st): ut_st e (ab, st): ab_st e (ab, tipab): ab_tipab e (tipab, serv): tipab_serv e
  - st.inizio <= pren.inizio < pren.fine < ab.fine()

$\forall\, ut,\ pren,\ serv\ (Utente(ut)\ \wedge\ Prenotazione(pren)\ \wedge\ Servizio(serv)\ \ \wedge\ \ ut_pren(ut,\ pren)\ \wedge\ pren_serv(pren,\ serv))$

$\Rightarrow\ \exists\, st,\ ab,\ tipab\ (Storico(st)\ \wedge\ Abbonamento(ab)\ \wedge\ TipoAbbonamento(tipab)$

$\wedge\ ut_st(ut,\ st)\ \wedge\ ab_st(ab,\ st)\ \wedge\ ab_tipab(ab,\ tipab)\ \wedge\ tipab_serv(tipab, serv)$

$\wedge\ \ (\exists\, iniz_{ab},\ fin_{ab}\ ,\ iniz_{pren},\ fin_{pren}$

$(\ inizio(ab,\ iniz_{ab})\ \wedge\ fine(ab,\ fin_{ab})$

$\wedge\ inizio(pren,\ iniz_{pren})\ \wedge\ fine(pren,\ fin_{pren}\ )$

$\wedge\ \ \leq\ (iniz_{ab},\ iniz_{pren})\ \ \wedge\ \ <\ (iniz_{pren},\ fin_{pren})\ \ \wedge\ \ <\ (fin_{pren}, fin_{ab}))$

[Postazione.Utente_uso_esclusivo]

Per ogni post: Postazione
non devono esistere st1: Storico e st2: Storico tali che

- (post, st1): post_st

- (post, st2): post_st
- st1 ≠ st2
- non_sovraposti(st1.inizio, st1.fine, st2.inizio, st2.fine) = False

$$\forall\, post\ \ Postazione(post)$$
$$\Rightarrow\ \neg\exists\, st1,\ st2\ (Storico(st1)\ \wedge\ Storico(st2)\ \wedge\ post_st(post,\ st1)\ \wedge\ post_st(post,\ st2)\ \wedge\ st1 \neq st2$$
$$\wedge\ (\ \exists\, iniz_{st1},\ fin_{st1},\ iniz_{st2},\ fin_{st2}$$
$$(DataOra(iniz_{st1})\ \wedge\ DataOra(fin_{st1})\ \wedge\ DataOra(iniz_{st2})\ \wedge\ DataOra(fin_{st2})$$
$$\wedge\ inizio(st1,\ iniz_{st1})\ \wedge\ fine(st1,\ fin_{st1})\ \wedge\ inizio(st2,\ iniz_{st1})\ \wedge\ fine(st2,\ fin_{st2})$$
$$\wedge\ non_sovraposti(iniz_{st1},\ fin_{st1},\ iniz_{st2},\ fin_{st2},\ False)$$

[Storico.inizio_fine_inclusi_in_durata_abbonamento]

Per ogni st: Storico tale che esiste ab: Abbonamento (ab, st): ab_st
deve essere vero che

- ad.inizio <= st.inizio < ab.fine() e st.fine non è valorizzato
  oppure
- ad.inizio <= st.inizio < st.fine <= ab.fine()

$$\forall\, st,\ ab\ \ (Storico(st)\ \wedge\ Abbonamento(ab)\ \wedge\ ab_st(ab,\ st)$$
$$\Rightarrow\ \exists\, iniz_{st},\ iniz_{ab},\ fin_{ab}(\ DataOra(iniz_{st})\ \wedge\ DataOra(iniz_{ab})\ \wedge\ DataOra(fin_{ab})$$
$$\wedge\ inizio(st,\ iniz_{ab})\ \wedge\ inizio(ab,\ iniz_{ab})\ \wedge\ fine(ab,\ fin_{ab})$$
$$\wedge\ \ \leq (iniz_{ab},\ iniz_{st})\ \wedge\ \ <(iniz_{st}, fin_{ab})$$
$$\wedge\ (\ \ (\neg\,\exists\, fin_{st}\,(DataOra(fin_{st})\ \ \wedge\ \ fine(st,\ fin_{st}))$$
$$\vee$$
$$(\ \exists\, fin_{st}\,(DataOra(fin_{st})\ \ \wedge\ \ fine(st,\ fin_{st})\ \wedge\ \ \leq (fin_{st},\ fin_{ab}))\ \ )$$

[AccessoSpazio.Utente_accede_solo_con_abbonamento_valido]

Per ogni asp: AccessoSpazio e ut: Utente tali che (ut, asp): ut_asp
deve essere vero che esiste st: Storico tale che

- (ut, st): ut_st
- (asp, st): accesso_con
- esiste ab: Abbonamento tale che (ab, st): ab_st
- esiste post: Postazione tale che (post, st): post_st
- st.inizio <= asp.entrata
- se st.fine è valorizzato allora asp.entrata < st.fine
  - se asp.uscita è valorizzato allora asp.uscita <= st.fine

$$\forall\, asp,\ ut\ (\ AccessoSpazio(asp)\ \ \wedge\ \ Utente(ut)\ \ \wedge\ ut_asp(ut,\ asp))$$
$$\Rightarrow\ \exists\, st\ (\ Storico(st)\ \wedge\ ut_st(ut,\ st)\ \wedge\ accesso_con(asp,\ st)$$
$$\wedge\ \exists\, ab\ (\ Abbonamento(ab)\ \wedge\ ab_st(ab,\ st)\ )$$
$$\wedge\ \exists\ post\ (\ Postazione(post)\ \wedge\ post_st(post,\ st)\ )$$
$$\wedge\ \exists\, iniz_{st},\ entr_{asp}\ (\ (DataOra(iniz_{st})\ \wedge\ DataOra(entr_{asp})$$
$$\wedge\ inizio(st,\ iniz_{st})\ \wedge\ \ entrata(asp,\ entr_{asp})\ \ \wedge\ \ \leq (iniz_{st},\ entr_{asp})\ \ )$$
$$\wedge\ ($$
$$(\ \neg\exists\, fin_{st}\ (\ (DataOra(fin_{st})\ \ \wedge\ fine(st,\ fin_{st})\ )$$

$$
\begin{array}{l}
\vee \\
(\exists\, fin_{st}\, (\, (DataOra(fin_{st}) \wedge fine(st, fin_{st}) \wedge < (ent_{asp}, fin_{st}) \\
\quad \wedge\, (\\
\qquad (\neg\exists\, usc_{asp}\, (DataOra(usc_{asp}) \wedge uscita(asp, usc_{asp}) \\
\qquad \vee \\
\qquad (\exists\, usc_{asp}\, (DataOra(usc_{asp}) \wedge uscita(asp, usc_{asp}) \wedge \leq (usc_{asp}, fin_{ab})\,) \\
\quad)\,) \\
)\,)
\end{array}
$$

[AcessoSpazio.Entrata_minore_Uscita]

Per ogni asp: AccessoSpazio deve essere vero che

- asp.entrata < asp.uscita

oppure

- asp.uscita NON è valorizzato

$$
\begin{array}{l}
\forall\, asp\, (AccessoSpazio(asp) \\
\quad \Rightarrow\, (\, (\, (\exists\, entr, usc\, (entrata(asp, entr) \wedge uscita(asp, usc)) \wedge < (entr, usc)) \\
\qquad \vee \\
\quad (\, (\exists\, entr\, (\, entrata(asp, entr)\,) \wedge \neg\exists\, usc\, (uscita(asp, usc))\,)
\end{array}
$$

[UsoServizio.Inizio_minore_Fine]

Per ogni userv: UsoServizio deve essere vero che

- userv.inizio < userv.fine

oppure

- userv.fine NON è valorizzato

$$
\begin{array}{l}
\forall\, userv\, (UsoServizio(userv) \\
\quad \Rightarrow\, (\, (\, (\exists\, iniz, fin\, (\, inizio(userv, iniz) \wedge fine(userv, fin)) \wedge < (iniz, fin)) \\
\qquad \vee \\
\quad (\, (\exists\, iniz\, (\, inizio(userv, iniz)\,) \wedge (\neg\exists\, fin\, (\, fine(userv, fin)\,)\,)
\end{array}
$$

[UsoServizio.Quantita_valorizzata_se_fine_valorizzato]

Per ogni userv: UsoServizio tale che userv.fine é valorizzatto deve essere vero che userv.quantita è valorizzata.

$$\forall\, userv\, (\, UsoServizio(userv) \wedge \exists\, fin\, (\, DataOra(fin) \wedge fine(userv, fin)\,) \Rightarrow \exists\, q\, (Intero(q) \wedge quantita(userv, q))$$

[UsoServizio.inizio_fine_inclusi_in_AccessoSpazio_esistente]

Per ogni userv: UsoServizio, asp: AccessoSpazio tali che (asp, userv): asp_userv

deve essere vero che

- asp.entrata <= userv.inizio e userv.fine NON è valorizzato e asp.uscita NON è valorizzato

oppure

- asp.entrata <= userv.inizio < userv.fine e asp.uscita NON è valorizzato

oppure

- asp.entrata <= userv.inizio < userv.fine < asp.uscita

$$\forall\, userv,\ asp\ (\ UsoServizio(userv)\ \wedge\ AcessoSpazio(asp)\ \wedge\ asp_userv(asp,\ userv)\ )$$

$$\Rightarrow\ (\exists\, entr_{asp},\ iniz_{userv}\,(entrata(asp,\ entr_{asp})\ \wedge\ inizio(userv,\ iniz_{userv}))\ \wedge\ \ \leq (entr_{asp}, iniz_{userv})$$

$$\wedge\ ($$

$$(\neg\exists\, fin_{userv}\,(fine(userv,\ fin_{userv})\ \wedge\ \neg\,\exists\, usc_{asp}\,(uscita(asp, usc_{asp}))$$

$$\vee$$

$$(\exists\, fin_{userv}\,(fine(userv,\ fin_{userv}))\ \wedge\ \neg\exists\, usc_{asp}\,(uscita(asp, usc_{asp}))\ \wedge\ \ < (iniz_{userv}, fin_{userv})\ )$$

$$\vee$$

$$(\exists\, fin_{userv}, usc_{asp}\,(fine(userv,\ fin_{userv})\ \wedge\ uscita(asp, usc_{asp})\ \wedge\ \ < (iniz_{userv},\ fin_{userv})\ \wedge\ \ < (fin_{userv},\ usc_{asp}))$$

[Prenotazione.Inizio_minore_Fine]

Per ogni pren: Prenotazione deve essere vero che

- pren.inizio < pren.fine

$$\forall pren\ (Prenotazione(pren)$$

$$\Rightarrow\ \exists\, iniz,\ fin\ (inizio(pren,\ iniz)\ \wedge\ fine(pren,\ fin)\ \wedge\ \ < (iniz,\ fin)\ )\ )$$

[Prenotazione.istante_canc_valorizzata_se_cancellata=True]

Per ogni pren: Prenotazione deve essere vero che

- pren.istante_canc è valorizzato se e solo se pren.cancellata = True

$$\forall\, pren\ (Prenotazione(pren) \Rightarrow (cancellata(pren,\ True)\ \Leftrightarrow \exists\, d\ (DataOra(d)\ \wedge\ istante_canc(pren,\ d))$$

[Servizio.se_prenotabile=True_allora_Prenotazione_obbligatoria_prima_UsoServizio]

Per ogni serv: Servizio, userv: UsoServizio, asp: AccessoSpazio, ut: Utente tali che

- (userv, serv): userv_serv
- (asp, userv): asp_userv
- (ut, asp): ut_asp
- serv.prenotabile = True

deve essere vero che

- esiste pren: Prenotazione tale che
  - (ut, pren): ut_pren
  - (pren, serv): pren_serv e
  - pren.cancellata = False
  - pren.istante_prenotaz < userv.inizio
  - pren.inizio <= userv.inizio < userv.fine <= pren.fine

$$\forall\, serv,\ userv,\ asp,\ ut\ \ (Servizio(serv)\ \wedge\ UsoServizio(userv)\ \wedge\ AccessoSpazio(asp)\ \wedge\ Utente(ut)$$

$$\wedge\ \ userv_serv(userv,\ serv)\ \wedge\ asp_userv(asp,\ userv)\ \wedge\ ut_asp(ut_asp)\ \wedge\ prenotabile(serv,\ True))$$

$$
\begin{aligned}
\Rightarrow\ & \exists\, pren\ (Prenotazione(pren)\ \wedge\ ut_pren(ut,\ pren)\ \wedge\ pren_serv(pren,\ serv)\ \wedge\ cancellata(pren,\ False) \\
& \quad \wedge\ \exists\, ist_{pren},\ iniz_{pren},\ fin_{pren},\ iniz_{userv}\ (istante_prenotaz(pren,\ ist_{pren}) \\
& \qquad \wedge\ inizio(pren,\ iniz_{pren})\ \wedge\ fine(pren, fin_{pren})\ \wedge\ inizio(userv,\ iniz_{userv}\ )) \\
& \qquad \wedge\ \ <(ist_{pren}, iniz_{userv}\ )\ \wedge\ \ \leq(iniz_{pren},\ iniz_{userv}\ ) \\
& \qquad \wedge\ \ (\ (\exists\, fin_{userv}\ (fine(userv,\ fin_{userv})\ \wedge\ \ \leq(fin_{userv},\ fin_{pren})) \\
& \qquad\qquad \vee \\
& \qquad\quad (\ (\neg\exists\, fin_{userv}\ (fine(userv,\ fin_{userv})\ )\ ))
\end{aligned}
$$

[Servizio.uso_esclusivo_durante_prenotazione]

Per ogni serv: Servizio, pren: Prenotazione, ut1: Utente tali che

- serv.prenatabile = True
- (pren, serv): pren_serv
- (ut1, pren): ut_pren
- pren.cancellata = False

deve essere vero che non esistono

- ut2: Utente, asp2: AccessoSpazio, userv2: UsoServizio tali che
  - ut2 ≠ ut1
  - (ut2, asp2): ut_asp
  - (asp2, userv2): asp_userv
  - (userv2, serv): userv_serv
  - se userv.fine è valorizzato
    - non_sovrapposti(pren.inizio, pren.fine, userv.inizio, userv.fine) = False
  - altrimenti
    - non_sovrapposti(pren.inizio, pren.fine, userv.inizio, adesso) = False

$$
\begin{aligned}
& \forall\, serv,\ pren,\ ut1\ (\ Servizio(serv)\ \wedge\ Prenotazione(pren)\ \wedge\ Utente(ut) \\
& \quad \wedge\ prenotabile(serv,\ True)\ \wedge\ pren_serv(pren,\ serv)\ \wedge\ ut_pren(ut,\ pren)\ \wedge\ cancellata(pren,\ False) \\
& \quad \Rightarrow\ \neg\exists\, ut2,\ asp2,\ userv2\ (\ Utente(ut2)\ \wedge\ AccessoSpazio(asp2)\ \wedge\ UsoServizio(userv2) \\
& \qquad \wedge\ ut2\ \neq\ ut1\ \wedge\ ut_asp(ut2,\ asp2)\ \wedge\ asp_userv(asp2,\ userv2)\ \wedge\ userv_serv(userv2,\ serv) \\
& \qquad \wedge\ \exists\, iniz_{pren},\ fin_{pren},\ iniz_{userv}\ (\ DataOra(iniz_{pren})\ \wedge\ DataOra(fin_{pren})\ \wedge\ DataOra(iniz_{userv}) \\
& \qquad\quad \wedge\ inizio(pren,\ iniz_{pren})\ \wedge\ fin(pren,\ fin_{pren})\ \wedge\ inizio(userv,\ iniz_{userv}) \\
& \qquad\quad \wedge\ (\\
& \qquad\qquad (\ \exists\ fin_{userv}\ (\ DataOra(fin_{userv})\ \wedge\ fine(userv,\ fin_{userv}) \\
& \qquad\qquad\quad \wedge\ non_sovrapposti(\ iniz_{pren},\ fin_{pren},\ iniz_{userv},\ fin_{userv},\ False)\ ) \\
& \qquad\qquad \vee \\
& \qquad\qquad (\ \neg\exists\ fin_{userv}\ (\ DataOra(fin_{userv})\ \wedge\ fine(userv,\ fin_{userv}) \\
& \qquad\qquad\quad \wedge\ \exists\ adesso\ (\ DataOra(adesso)\ ) \\
& \qquad\qquad\quad \wedge\ non_sovrapposti(\ iniz_{pren},\ fin_{pren},\ iniz_{userv}, adesso,\ False)\ ) \\
& \qquad\quad)\ \ )
\end{aligned}
$$

[TipoAbbonamento.Inizio_iscrizione_minore_Fine_iscrizione]

Per ogni tipab: TipoAbbonamento deve essere

- tipap.inizio_iscrizione < tipap.fine_iscrizione

$$
\forall\, tipab\ (TipoAbbonamento(tipab)
$$

$\Rightarrow \exists\, iniz,\ fin\ \ (inizio_iscrizione(tipab,\ iniz)\ \wedge\ fine_iscrizione(tipab,\ fin)\ \wedge\ \ <(iniz,\ fin)$

[TipoAbbonamento.num_Utenti_associati_Abbonamento_minoreUguale_max_num_utenti]

Per ogni tipab: TipoAbbonamento, ab: Abbonamento tali che (ab, tipab): ab_tipab

sia N insieme di ut: Utente tali che esiste st: Storico tale che (ut, st): ut_st e (ab, st): ab_st.

deve essere vero che

- |N| <= tipab.max_num_utenti

$\forall\, tipab,\ ab\ (TipoAbbonamento(tipab)\ \wedge\ Abbonamento(ab)\ \wedge\ ab_tipab(ab,\ tipab)$

$\wedge\ \exists\, num\ (Intero(num)\ \wedge\ max_num_utenti(tipab,\ num))$

$\Rightarrow\ \ |\{ut\,|\,Utente(ut)\ \wedge\ \exists st\ \ (Storico(st)\ \wedge\ ut_st(ut,\ st)\ \wedge\ ab_st(ab,\ st))\}|\ \ \leq\ \ num$

Operazioni di classe *UsoServizio*

Operazioni ausiliarie

calcolo_quantita_da_tempo(): Intero > 0

Precondizioni:

- this.fine è valorizzato
- esiste *serv*: Servizio
  - tale che *(this, serv): userv_serv*
- esiste *tipserv*: TipoServizio
  - tale che *(serv, tipserv): serv_tipserv* e *tipserv.quantita_da = tempo*
- esiste *um*: UnitaMisura
  - tale che *(tipserv, um): tipserv_um* e *um.nome* ∈ {secondo, minuto, ora, giorno, settimana}

Postcondizioni:

- operazione non fa slide effect
- viene restituiro valore "result" definito come segue
  - sia *n_sec* è il numero secondi tra *this.inizio* e *this.fine*
  - sia unita_sec = *this.secondi_per_unita_misura(um)*
  - sia X tale che
    - X * *unita_sec* >= *n_sec*
    - (X - 1) * *unita_sec* < *n_sec*
- result = X

Pre:

$\exists\, fin\ (\,DataOra(fin)\ \wedge\ \ fine(this,\ fin))$

$\wedge\ \ \exists\, serv\ (\,Servizio(serv)\ \wedge\ userv_serv(this,\ serv))$

$\wedge\ \ \exists\, tipserv\ (\,TipoServizio(tipserv)\ \ \wedge\ serv_tipserv(serv,\ tipserv)\ \wedge\ quantita_da(tipserv,\ tempo))$

$\wedge\ \ \exists\, um\ (\,UnitaMisura(um)\ \wedge\ tipserv_um(tipserv,\ um)$

$\wedge\ \ (nome(um,\ secondo)\ \ \vee\ nome(um,\ minuto)\ \vee\ nome(um,\ ora)\ \vee\ nome(um, giorno)\ \vee\ nome(um,\ settimana)$

Post:

$\exists\, n_sec\ (\,Intero(n_sec)\ \ \wedge$ è il numero secondi tra *this.inizio* e *this.fine* ← non so come scrivere in FOL

$\exists\, unita_sec\ (\,Intero(unita_sec\,)\ \wedge\ secondi_per_unita_misura(this,\ um,\ unita_sec)$

$\exists\, X\ (\,Intero(X)\ \ \wedge\ \ \leq(n_sec,\ X\ *\ unita_sec)\ \wedge\ \ <(n_sec,\ (X-1)\ *\ unita_sec$

result = X

→ chiusura_UsoServizio(.)

Operazioni di classe

prezzo(): Reale >= 0

Precondizioni:

- *this.fine* è valorizzato
- *this.quantita* è valorizzato

Postcondizioni:

- operazione non fa slide effect
- viene restituiro valore "result" definito come segue
    - esiste *serv*: Servizio tale che *(this, serv): userv_serv*
    - esiste *asp*: AccessoSpazio tale che *(asp, this): asp_userv*
    - esiste *ut*: Utente tale che *(ut, asp): ut_asp*
    - *sia st: Storico tale che*
        - *(ut, st): ut_st*
        - *(asp, st): accesso_con*
        - esiste *ab*: Abbonamento tale che (ab, st): ab_st
        - esiste post: Postazione tale che (post, st): post_st
    - esiste *tipab*: TipAbbonamento tale che *(ab, tipab): ab_tipab e (tipab, serv): tipab_serv*
    - sia soglia_gratis_rimasta = ab.ab_quantita_serv_gratis_rimasta(serv, userv.inizio)
    - sia a_pagamento = *this.quantita* - soglia_gratis_rimasta
    - se a_pagamento <= 0
        - result = 0
    - altrimenti
        - result = a_pagamento * *serv.prezzo* * (1 - *(serv, tipab).sconto)*

→ costo_totale_servizi_oltre_soglia_gratis_nel_periodo(.)

→ Diagramma UML

Operazioni di classe *Abbonamento*

Operazioni ausiliarie

ab_quantita_serv_utilizata_in_mese_entro_data(serv: Servizio, t: DataOra): Intero >= 0

Precondizioni:

- esiste tipab: TipAbbonamento tale che (this, tipab): ab_tipab
- esiste (tipab, serv): tipab_serve
- this.inizio <= t < this.fine()

Postcondizioni:

- operazione non fa side effect
- viene restituito il valore result definito come segue
    - sia soglia_gratis = (tipab, serv).soglia_mensile_gratis
    - sia mese_t = mese di t
    - sia U insieme di userv: UsoServizio tali che
        - (userv, serv): userv_serv
        - mese di userv.inizio = mese_t
        - userv.fine < t
        - esiste asp: AccessoSpazio tale che (asp, userv): asp_userv e (asp, this): accesso_con
    - sia $sum_q = \sum_{userv \in U} userv.quantita$

result = sum_q

ab_quantita_serv_utilizata_in_mese(serv: Servizio, t: DataOra): Intero >= 0

Precondizioni:

- esiste tipab: TipAbbonamento tale che (this, tipab): ab_tipab
- esiste (tipab, serv): tipab_serve
- this.inizio <= t < this.fine()

Postcondizioni:

- operazione non fa side effect
- viene restituito il valore result definito come segue
  - sia mese_t = mese di t
  - sia anno_t = anno di t
  - sia U insieme di userv: UsoServizio tali che
    - (userv, serv): userv_serv
    - mese di userv.inizio = mese_t
    - anno di userv.inizio = anno_t
    - userv.fine < t
  - sia $sum_q = \sum_{userv \in U} userv.quantita$

result = sum_q

→ Diagramma UML

ab_quantita_serv_gratis_rimasta(serv: Servizio, t: DataOra): Intero >= 0

Precondizioni:

- esiste tipab: TipAbbonamento tale che (this, tipab): ab_tipab
- esiste (tipab, serv): tipab_serve
- this.inizio <= t < this.fine()

Postcondizioni:

- operazione non fa side effect
- viene restituito il valore result definito come segue
  - sia soglia_gratis = *(tipab, serv).soglia_mensile_gratis*
  - sia num = soglia_gratis - *this.ab_quantita_serv_utilizata_in_mese(serv, t)*
    - se num < 0,
      - return 0
    - altrimenti
      - return num

→ prezzo()

Operazioni di classe

fine(): DataOra

Precondizione: nessuna

Postcondizione:

- operazione non fa side effect
- viene restituito il valore “result” definito cosi
  - sia *tipab*: TipoAbbonamento tale che (this, tipab): ab_tipab
  - sia *um*: UnitaMisure tale che (tipab, um): tipab_um
  - sia *fine* = this.*inizio* + tipab.*durata* in unita um.*nome*

result = fine

$$\exists\, tipab,\ um,\ iniz,\ num,\ unit\ (\ TipoAbbonamento(tipab)\ \wedge\ ab_tipab(this,\ tipab)$$
$$\wedge\ UnitaMisura(um)\ \wedge\ tipab_um(tipab,\ um)$$
$$\wedge\ DataOra(iniz)\ \wedge\ inizio(this,\ iniz)$$
$$\wedge\ Intero(num)\ \wedge\ durata(tipab,\ num)$$
$$\wedge\ Stringa(unit)\ \wedge\ nome(um,\ unit)$$

$result\ =\ iniz\ +\ num\ (in\ unit)$ come si scrive?

→ verifica_abbonamento_valido(.)

→ [Utente.Prenota_nel_periodo_validita_suo_abbonamento]

utenti(): Utente [0..*]

Precondizione: nessuna
Postcondizione:

- operazione non fa side effect
- viene restituito il valore result definito come segue
    - sia U l'insieme di ut: Utenti tali che esiste st: Storico tale che (ut, st): ut_st e (this, st): ab_st

result = U

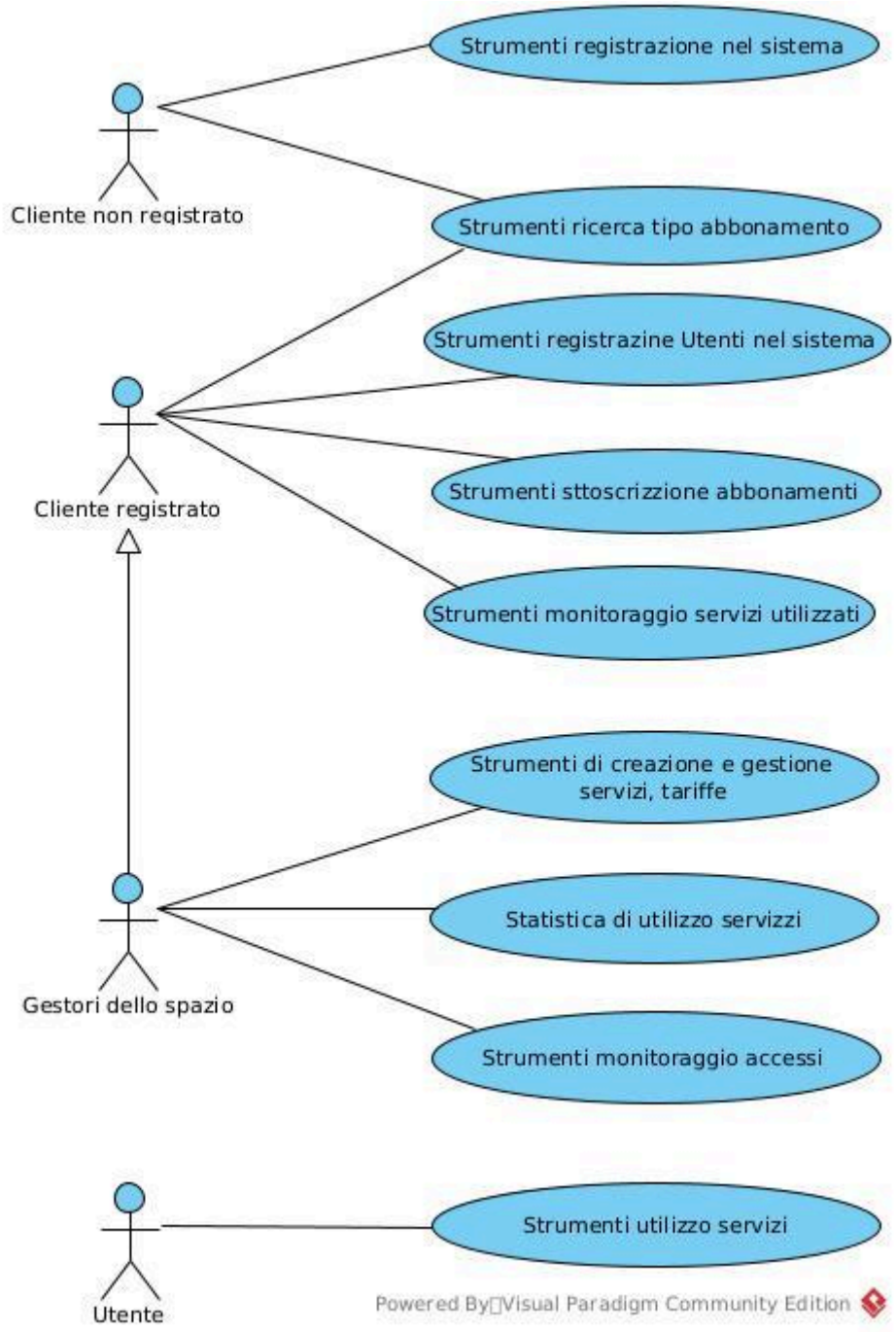
$$\forall\, ab\ (Abbonamento(ab))$$
$$\exists\, U\ =\ \{ut \mid Utente(ut)\ \wedge\ \exists st\ \ (Storico(st)\ \wedge\ ut_st(ut,\ st)\ \wedge\ ab_st(ab,\ st))\}$$

result = U

→ servizi_utilizzati(.)
→ utenti_associati_abbonamento_senza_entrate(.)

→ Diagramma UML

1.2. Diagramma UML degli Use-Case



Operazioni di Use-Case Statistica di utilizzo servizi

1. Il management dello spazio, per monitorare l'utilizzo delle postazioni, deve poter calcolare, dato un insieme di intervalli orari disgiunti (ad esempio, "8:00–12:30", "13:00–17:00", "19:00–23:00") la media giornaliera di utenti abbonati che hanno usufruito dello spazio per ogni intervallo orario, nel mese corrente.

media_giornaliera_utenti_nei_periodi(

O: Orario[1..*]

) : (Orario, Reale >= 0) [1..*]

Precondizioni: nessuna

Postcondizioni:

- operazione non fa side effect
- viene restituito il valore "result" definito cosi
  - sia mese_corrente = mese di adesso
  - sia anno_corrente = anno di adesso
  - sia D insieme dei d: Data tali che
    - d.mese = mese_corrente
    - d.anno = anno_corrente
    - d <= adesso
  - **per ogni or in O**
    - per ogni d in D
      - sia U insieme di ut: Utenti tali che
        - esiste st: Storico e ab: Abbonamento tale che
          - (ut, st): ut_st e (ab, st): ab_st
          - st.inizio.Data <= d
            e
          - se st.fine è valorizzato
            - d < st.fine.Data
          - altrimenti
            - d < ab.fine().Data
        - esiste almeno uno asp: AccessoSpazio tale che
          - (ut, asp): ut_asp
          - (asp, st): accesso_con
          - asp.entrata.data = d
          - se asp.uscita è valorizzato
            - sovrapposti_Ora(asp.entrata.ora, asp.uscita.ora,
              or.ora_inizio, or.ora_fine) = True
          - altrimenti
            - sovrapposti_Ora(asp.entrata.ora, adesso.ora,
              or.ora_inizio, or.ora_fine) = True
    - sum = $\sum_{d \in D} |U|$
    - media_or = sum / |D|
    - result = (or, media_or)

→ Diagramma UML

2. Il sistema deve poter calcolare, data un'impresa e un lasso di tempo, il costo totale dei servizi utilizzati oltre le soglie gratuite dagli utenti a essa associati, nel periodo indicato.

costo_totale_servizi_oltre_soglia_gratis_nel_periodo(

imp: Impresa,

iniz: DataOra,

fin: DataOra

) : Reale >= 0

Precondizioni:

- esiste almeno uno ab: Abbonamento tale che (imp, ab): stipula e ab.inizio <= iniz < fin <= ab.fine()

Postcondizioni:

- operazione non fa side effect
- viene restituito il valore “result” definito cosi
  - sia A insieme di ab: Abbonamento tali che
    - (imp, ab): stipula
    - non_sovrapposti(ab.inizio, ab.fine(), iniz, fin) = False
  - sia AS insieme di asp: AccessoSpazio tali che (asp, ab) dove ab in A
  - sia US insieme di userv: UsoServizio tali che
    - (asp, userv): asp_serv dove asp in AS
    - userv.prezzo() > 0
  - costo_totale = $\sum_{userv \in US} userv.prezzo()$

  result = costo_totale

3. Il management vuole poter calcolare, dato un periodo ed un intero positivo k,
i k servizi maggiormente utilizzati dagli utenti che hanno effettuato accessi nel periodo.

servizi_popolari_nel_periodo(

k: Intero > 0,

iniz: DataOra,

fin: DataOra

) : Servizio [0..*]

Precondizioni:

- iniz < fin

Postcondizioni:

- operazione non fa side effect
- viene restituito il valore “result” definito cosi
  - sia AC insieme di *asp*: AccessoSpazio tali che
    - *iniz <= asp.entrata < fin*
  - sia US insieme di *userv*: UsoServizio tali che
    - *(asp, userv): asp_userv* dove *asp* ∈ AC
  - sia N insieme delle coppie (serv, n) tali che
    - *serv*: Servizio tale che *(userv, serv): userv_serv* dove *userv* in US
    - *n* è il numero di link *userv_serv* ai quali partecipa *serv*
  - sia M insieme di k copie (serv, n) dal N tali che
    - per ogni copia (serv, n) di M non esiste (serv’, n’) in N dove n’ > n
  - sia Ms insieme di *serv* tali che *serv* risulta in copie (serv, n) in M

  result = Ms

→ Diagramma UML

Operazioni di Use-Case *Strumenti monitoraggio servizi utilizzati*

4. Il servizio clienti vuole poter calcolare, dato un periodo, gli utenti associati ad un abbonamento valido che non hanno effettuato alcun accesso nel periodo dato.

utenti_associati_abbonamento_senza_entrate(

ab: Abbonamento,

iniz: DataOra,

fin: DataOra

) : Utente[0..*]

Precondizioni:

- iniz >= ab.inizio e fin <= ab.fine()

Postcondizioni:

- Operazione non fa side effect
- Viene restituito il valore *result* definito come segue
  - sia U insieme di ut: Utente tali che U = ab.utenti()
  - sia U1 insieme di ut: Utenti tali che
    - (ut, asp): ut_asp dove ut in U e asp: AccessoSpazio tale che
      - iniz <= asp.entrata < fin
      - esiste (asp, st): accesso_con, dove st: Storico tale che esiste (ut, st): ut_st e (ab, st): ab_st
  - sia U2 = U / U1

result = U2

I clienti devono poter monitorare i servizi utilizzati dagli utenti associati ai loro abbonamenti.

servizi_utilizzati(

cl: Cliente,

A: Abbonamento[1..*],

iniz: DataOra,

fin: DataOra

) : (Abbonamento, Servizio [0..*])

Precondizioni:

- esiste link (cl, ab): stipula
- ab.inizio <= iniz < fin <= ab.fin()

Postcondizioni:

- Operazione non fa side effect
- Viene restituito il valore *result* definito come segue
  - sia N insieme delle coppie (ab, S_serv) tali che
    - ab in A
    - S_serv viene calcolato come segue:
      - sia U = ab.utenti()
      - sia AS insieme di *asp*: AccessoSpazio tali che (ut, asp): ut_asp , dove ut in U
      - sia US insieme di *userv*: UsoServizio tali che (asp, userv): asp_userv, dove asp in AS
      - sia S insieme di *serv*: Servizio tali che (userv, serv): userv_serv, dove userv in US
      - S_serv = S
  - per ogni ab in A esiste coppia (ab, S_serv) in N

result = N

→ Diagramma UML

Operazioni di Use-Case *Strumenti utilizzo servizi*

gestisce valorizzazione attributi UsoServizio.fine e UsoServizio.quantita

chiusura_UsoServizio(

userv: UsoServizio,

fin: DataOra [0..1],

quantita_da_sist_esterno: Intero > 0 [0..1])

Precondizione:

- *userv.fine* non è valorizzato
- *userv.quantita* non è valorizzato
- esiste *serv*: Servizio tale
    - che *(userv, serv): userv_serv*
- esiste *tipserv*: TipoServizio
    - tale che *(serv, tipserv): serv_tipserv*
- se *tipserv.quantita_da* = *sistema_esterno* allora
    - **quantita_da_sist_esterno** deve essere valorizzato

Postcondizione:

- esiste *serv*: Servizio
    - tale che *(userv, serv): userv_serv*
- esiste *tipserv*: TipoServizio
    - tale che *(serv, tipserv): serv_tipserv*
- esiste *um*: UnitaMisura
    - tale che *(tipserv, um): tipserv_um*
- se **fin** è valorizzato
    - allora *userv.fine* = *fin*
- se **fin** non è valorizzato
    - allora *userv.fine* = istante di adesso
- se *tipserv.quantita_da* = *sistema_esterno*
    - allora *userv.quantita* = ***quantita_da_sist_esterno***
- se *tipserv.quantita_da* = tempo
    - allora *userv.quantita* = *userv.calcolo_quantita_da_tempo()*

Gli utenti devono poter effettuare l'accesso, registrare l'uscita.

registrazione_accesso_spazio(ut: Utente): AccessoSpazio

Precondizioni:

- esiste st: Storico, post: Postazione, ab: Abbonamento tali che
    - (ut, st): ut_st
    - (ab, st): ab_st
    - (post, st): post_st
    - st.inizio <= adesso
    - adesso < ab.fine()
- non esiste asp': AccessoSpazio tale che
    - (ut, asp'): ut_asp
    - asp'.uscita non è valorizzato

Postcondizioni:

- viene creato e restituito oggetto result: AccessoSpazio con valori
    - result.entrata = adesso
    - viene creato il link (ut, result): ut_asp
    - viene creato il link (result, st): accesso_con

registrazione_uscita_spazio(asp: AccessoSpazio, ut: Utente)

Precondizioni:

- esiste link (ut, asp): ut_asp
- asp.fine non è valorizzato

Postcondizioni:

- viene valorizzato il valore asp.fine = adesso

→ Diagramma UML

Operazioni di Use-Case *Strumenti ricerca tipo abbonamento*

Potenziali clienti devono poter scegliere e sottoscrivere un abbonamento di una tipologia disponibile nel periodo corrente.

ricerca_TipoAbbonamento_attuale_con_Servizio(serv: Servizio) : TipoAbbonamento [0..*]

Precondizioni: nessuna
Postcondizioni:

- Operazione non fa side effect
- Viene restituito il valore *result* definito come segue
    - sia TA insieme di tipab: TipoAbbonamento tali che
        - (tipab, serv): tipab_serv
        - tipab.inizio_iscrizione <= adesso < tipab.fine_iscrizione

  result = TA

Operazioni di Use-Case *Strumenti sottoscrizione abbonamenti*

sottoscrizione_abbonamento_con_TipoAbbonamento(cl: Cliente, tipab: TipoAbbonamento, U: Utenti[1..*])

Precondizioni:

- tipab.inizio_iscrizione <= adesso < tipab.fine_iscrizione
- |U| <= tipab.max_num_utenti
- sia P insieme di post: Postazione tali che non esiste nessun st': Storico tale che (post, st'): post_st
- |P| <= |U|

Postcondizioni:

- viene creato e restituito oggetto result: Abbonamento con valori:
    - result.inizio = istante di adesso
    - viene creato il link (result, tipab): ab_tipab
    - viene creato il link (result, cl): stipula
    - per ogni ut in U
        - viene creato oggetto st: Storico con valori:
            - st.inizio = adesso
            - viene creato il link (result, st): ab_st
            - viene creato il link (ut, st): ut_st
            - viene creato il link (post, st): post_st, dove post in P

Operazioni di Use-Case *Strumenti monitoraggio accessi*

I gestori dello spazio devono poter verificare l'abbonamento di un utente. (autorizzazione entrata)

verifica_abbonamento_valido(

ut: Utente,

ab: Abbonamento

) : Boolean

Precondizioni: nessuna

Postcondizioni:

- result = True

  se e solo se
    - esiste st: Storico tale che
        - (ut, st): ut_st
        - (ab, st): ab_st
        - st.inizio <= adesso < ab.fine()

→ Diagramma UML

Operazioni di Use-Case *Strumenti di creazione e gestione servizi, tariffe*

I gestori dello spazio devono poter creare nuove tariffe

creazione_TipoAbbonamento(

nom: Stringa,
prez: Intero > 0,
durata: Intero > 0,
max_ut: Intero > 0,
iniz: DataOra,
fin: DataOra,
um: UnitaMisura
) : TipoAbbonamento

Precondizioni:

- non esiste tipab: TipoAbbonamento tale che tipab.nome = nom

Postcondizioni:

- viene creato e restituito oggetto result: TipoAbbonamento con valori:
  - result.nome = nom
  - result.prezzo = prez
  - result.durata = durata
  - result.max_num_utenti = max_ut
  - result.inizio_iscrizione = iniz
  - result.fine_iscrizione = fin
  - viene creato il link (result, um): tipab_um

creazione_tariffa(

tipab: TipoAbbonamento,
serv: Servizio,
soglia_gratis: Intero > 0,
sconto: Reale in 0..1)

Precondizioni:

- non esiste link (tipab, serv): tipab_serv

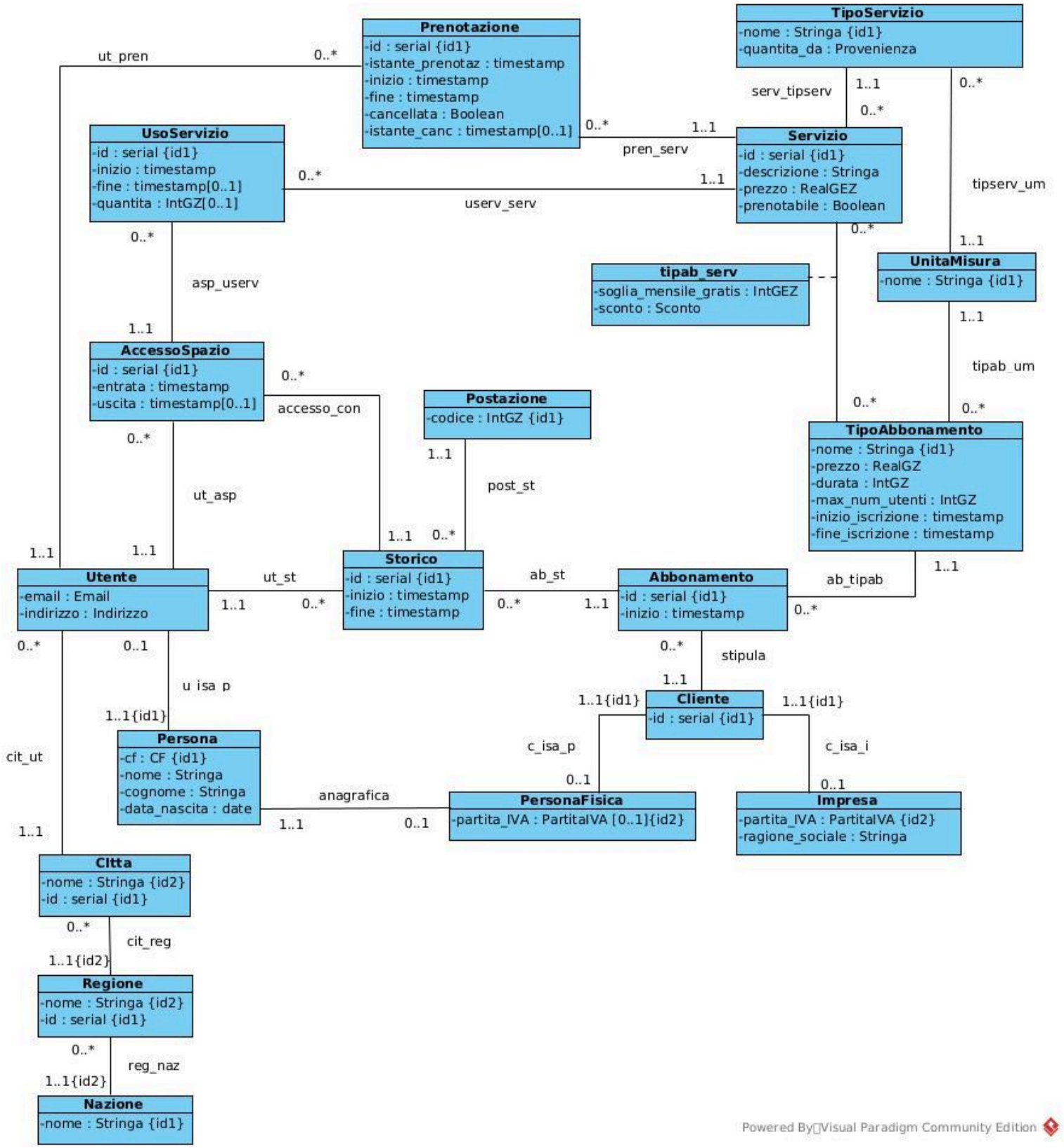
Postcondizioni:

- viene creato il link (tipab, serv): tipab_serv con valori:
  - (tipab, serv).soglia_mensile_gratis = soglia_gratis
  - (tipab, serv).sconto = sconto

→ Diagramma UML

2. Ristrutturazione per Basi Dati

2.1. Diagramma UML ristrutturato per bd



2.2. Specifica dei vincoli esterni

[Cliente.Disgiunzione]

Per ogni cl: Cliente
non devono esiste persf: PersonaFisica e im: Impresa tali che
(cl, persf): c_isa_p e (cl, im): c_isa_i

[Cliente.Completezza]

Per ogni cl: Cliente
esiste persf: PersonaFisica tale che (cl, persf): c_isa_p
oppure
esiste im: Impresa tale che (cl, im): c_isa_i

2.3. Traduzione diretta

```
begin transaction;

create type provenienza as enum('tempo', 'sistema esterno');

create domain stringa as varchar;

create domain intgz as integer check (value > 0);

create domain intgez as integer check (value >= 0);

create domain realgz as numeric check (value > 0);

create domain realgez as numeric check (value >= 0);

create domain cf as char(16)
   check (value ~ '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$');

create domain email as varchar(100)
   check (value ~
'^[a-z0-9!#$%&''*+/=?^_`{|}~-]+(\.[a-z0-9!#$%&''*+/=?^_`{|}~-]+)*@([a-z0-9]([a-z0-9-]*[a-z
0-9])?\.)+[a-z0-9]([a-z0-9-]*[a-z0-9])?$');

create domain sconto as numeric
   check (value BETWEEN 0 and 1);

create domain stringa_not_null as
   varchar(100) check(value is not null);

create domain cap_not_null as
   char(5) check (value is not null and value ~ '^[0-9]{5}$');
create type indirizzo as (
   via     stringa_not_null,
   civico  stringa_not_null,
   cap      cap_not_null
);

create domain partitaiva as char(11)
   check (value ~ '^[0-9]{11}$');



create table nazione (
```

```sql
   nome stringa primary key
);

create table regione (
   id serial primary key,
   nome stringa not null,
   nazione stringa not null,  -- accorpa naz_reg
   unique(nome, nazione),
   foreign key (nazione)
       references nazione(nome)
);

create table citta (
   id serial primary key,
   nome stringa not null,
   regione integer not null,   -- accorpa reg_cit
   unique(nome, regione),
   foreign key (regione)
       references regione(id)
);

create table persona (
   cf cf primary key,
   nome stringa not null,
   cognome stringa not null,
   data_nascita date not null
);

create table utente (
   persona cf primary key,
   email email not null,
   indirizzo indirizzo not null,
   citta integer not null,  -- accorpa cit_ut
   foreign key (persona)
       references persona(cf),
   foreign key (citta)
       references citta(id)
);

create table cliente (
   id serial primary key
);

create table impresa (
   cliente integer primary key,
   partita_iva partitaiva not null,
   unique (partita_iva),
   ragione_sociale stringa not null,
   foreign key (cliente)
       references cliente(id)
```

```sql
);

create table personafisica (
   cliente integer primary key,
   partita_iva partitaiva,
   unique (partita_iva),
   persona cf not null,     --  accorpa anagrafica
   unique (persona),        -- non possono esistere due personefisiche diverse riferirsi
alla stessa persona
   foreign key (cliente)
       references cliente(id),
   foreign key (persona)
       references persona(cf)
);

create table unitamisura (
   nome stringa primary key
);

create table tipoabbonamento (
   nome stringa primary key,
   prezzo realgez not null,
   durata intgz not null,
   max_num_utenti intgz not null,
   inizio_iscrizione timestamp not null,
   fine_iscrizione timestamp not null,
   check (fine_iscrizione > inizio_iscrizione),
   unitamisura stringa not null,        -- accorpa tipab_um
   foreign key (unitamisura)
       references unitamisura(nome)
);

create table abbonamento (
   id serial primary key,
   inizio timestamp not null,
   tipoabbonamento stringa not null,  -- accorpa ab_tip
   cliente integer not null,          -- accorpa stipula
   foreign key (tipoabbonamento)
       references tipoabbonamento(nome),
   foreign key (cliente)
       references cliente(id)
);

create table postazione (
   codice intgz primary key
);

create table Storico (
   id serial primary key,
   inizio timestamp not null,
```

```
   fine timestamp,
   check (fine is null or inizio < fine),
   utente cf not null,             -- accorpa ut_st
   abbonamento integer not null,   -- accorpa ab_st
   postazione intgz not null,      -- accorpa post_st
   foreign key (utente)
       references utente(persona),
   foreign key (abbonamento)
       references abbonamento(id),
   foreign key (postazione)
       references postazione(codice)
);

create table tiposervizio (
   nome stringa primary key,
   quantita_da provenienza not null,
   unitamisura stringa not null,
   foreign key (unitamisura)
       references unitamisura(nome)
);

create table servizio (
   id serial primary key,
   descrizione stringa not null,
   prezzo realgez not null,
   prenotabile boolean not null,
   tiposervizio stringa not null,   -- accorpa serv_tip
   foreign key (tiposervizio)
       references tiposervizio(nome)
);

create table tipab_serv (
   servizio integer not null,
   tipoabbonamento stringa not null,
   primary key (servizio, tipoabbonamento),
   soglia_mensile_gratis intgez not null,
   sconto sconto not null,
   foreign key (servizio)
       references servizio(id),
   foreign key (tipoabbonamento)
       references tipoabbonamento(nome)
);

create table accessospazio (
   id serial primary key,
   entrata timestamp not null,
   uscita timestamp,
   utente cf not null,           -- accorpa ut_asp
   storico integer not null, -- accorpa accesso_con
   check (uscita is null or uscita > entrata),
```

```sql
   foreign key (utente)
       references utente(persona),
   foreign key (storico)
       references storico(id)
);

create table usoservizio (
   id serial primary key,
   inizio timestamp not null,
   fine timestamp,
   quantita intgz,
   check (
       (fine is null and quantita is null)
       or
       (fine is not null and quantita is not null and fine > inizio)
   ),
   accessospazio integer not null,  -- accorpa asp_serv
   servizio integer not null,       -- accorpa aserv_serv
   foreign key (accessospazio)
       references accessospazio(id),
   foreign key (servizio)
       references servizio(id)
);

create table prenotazione (
   id serial primary key,
   istante_prenotazione timestamp not null,
   inizio timestamp not null,
   fine timestamp not null,
   check (fine > inizio),
   cancellata boolean not null,
   istante_canc timestamp,
   check (
       (cancellata = false and istante_canc is null)
       or
       (cancellata = true and istante_canc is not null)
   ),
   utente cf not null,         -- accorpa ut_pren
   servizio integer not null,  -- accorpa pren_serv
   foreign key (utente)
       references utente(persona),
   foreign key (servizio)
       references servizio(id)
);


commit;
```